

Laporan Praktikum Kontrol Cerdas

Mediapipe Hands

Nama : Achmad Rizky H.W
NIM : 234308059
Kelas : TKA-6C
Akun Github : <https://github.com/rizkywiyoso05-max>

Abstract

Praktikum ini membahas implementasi deteksi dan pelacakan tangan secara real-time menggunakan MediaPipe Hands dan OpenCV pada bahasa pemrograman Python. Sistem dirancang untuk mendeteksi keberadaan tangan, menampilkan 21 titik landmark, mengidentifikasi tangan kanan dan kiri, serta menentukan orientasi tangan (telapak atau punggung). Proses deteksi dilakukan melalui pengambilan citra webcam yang dikonversi dari format BGR ke RGB sebelum diproses oleh model MediaPipe. Hasil percobaan menunjukkan bahwa sistem mampu bekerja secara real-time dengan tingkat akurasi yang baik pada kondisi pencahayaan yang memadai. Landmark yang dihasilkan memungkinkan analisis struktur dan pergerakan tangan secara detail sehingga dapat dikembangkan lebih lanjut untuk aplikasi berbasis gestur, augmented reality, dan interaksi manusia-komputer.

Kata kunci: MediaPipe Hands, OpenCV, hand tracking, visi komputer.

1. Pendahuluan

Mediapipe merupakan sebuah framework yang digunakan untuk membuat pipeline yang digunakan untuk membuat pipeline yang akan digunakan pada machine learning untuk pemrosesan data dalam bentuk video, audio, dsb. Teknologi ini mampu mengenali 21 titik landmark pada tangan sehingga dapat digunakan untuk mengidentifikasi posisi tangan kanan dan kiri serta orientasi telapak dan punggung tangan. MediaPipe Hands banyak digunakan karena akurat, ringan, dan mudah diimplementasikan dengan bahasa pemrograman Python.

2. Tujuan

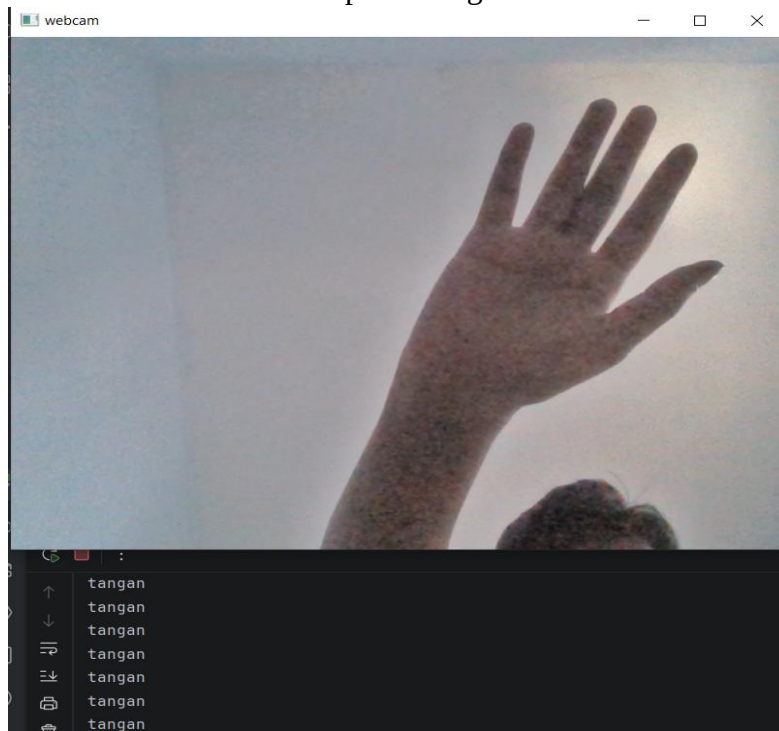
- Memahami konsep dasar deteksi dan pelacakan tangan menggunakan MediaPipe Hands.
- Menerapkan konsep visi komputer dan machine learning dalam pemrograman Python.
- Meningkatkan pemahaman mahasiswa terhadap penerapan visi komputer dan machine learning.

3. Manfaat

- Melatih kemampuan pemrograman dan logika analisis berbasis kamera.
- Mendukung pengembangan aplikasi interaktif seperti pengendalian sistem berbasis gestur, augmented reality, dan virtual reality.
- Menjadi media pembelajaran yang efektif dalam memahami penerapan visi komputer dan kecerdasan buatan.

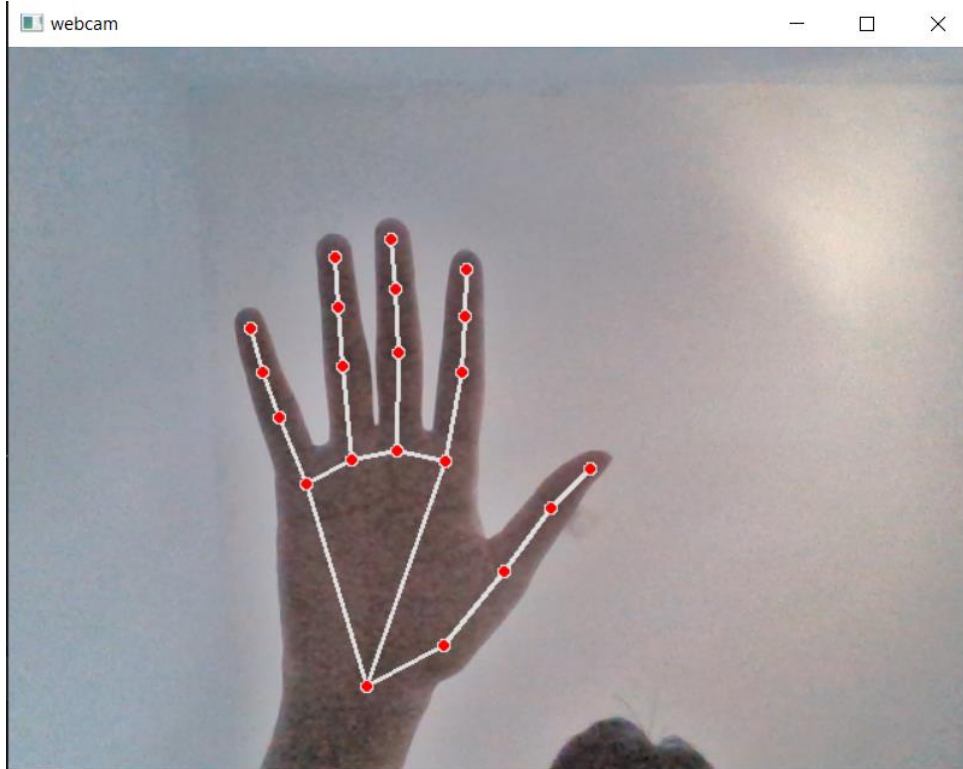
4. Hasil Program

A. Percobaan A Deteksi Tampilan Tangan



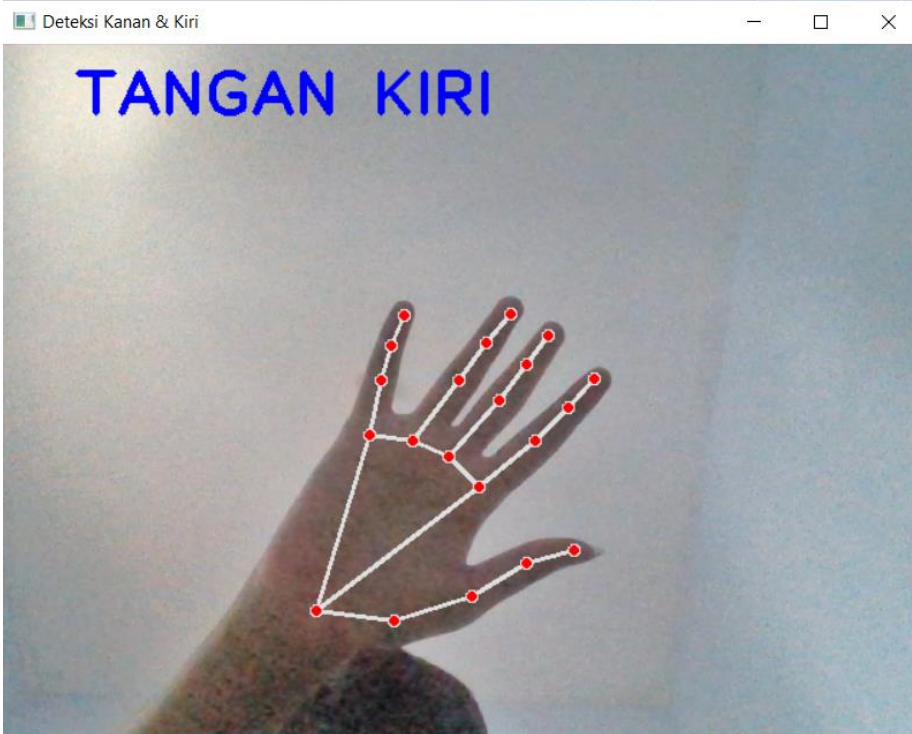
```
Percobaan A.py x
1 import cv2 #import module openCV
2 import mediapipe
3
4 capture = cv2.VideoCapture(0) #video capture pada device kamera nomer 1
5 mediapipehand = mediapipe.solutions.hands
6 tangan = mediapipehand.Hands() #variable tangan untuk menyimpan konfigurasi deteksi tangan
7
8 while True:
9     success, img = capture.read() #menyimpan citra tangkapan kamera ke img
10    imgRGB = cv2.cvtColor(img,cv2.COLOR_BGR2RGB) #merubah warna img ke RGB
11    result=tangan.process(imgRGB) #melakukan pemrosesan dari citra imgRGB
12    if result.multi_hand_landmarks:
13        print("tangan") #ketika tangan terdeteksi menampilkan "tangan" pada terminal
14    else:
15        print("tidak ada") #ketika tangan terdeteksi menampilkan "tidak ada" pada terminal
16
17    cv2.imshow("webcam",img)
18    cv2.waitKey(10)
19    if cv2.waitKey(10) & 0xFF == ord('q'):
20        break
21
22 capture.release() # Tutup webcam dan jendela tampilan saat 1 ditekan
23 cv2.destroyAllWindows()
24
25
26
```

B. Percobaan B Hands Landmarks



```
Percobaan A.py  Percobaan B.py x
1  import cv2 #import module opencv
2  import mediapipe
3  capture = cv2.VideoCapture(0)
4  mediapipehand = mediapipe.solutions.hands
5  tangan = mediapipehand.Hands(max_num_hands=1)
6  mediapipe draw = mediapipe.solutions.drawing_utils
7
8  while True:
9      success, image = capture.read() #read video frame by frame
10     imgRGB = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
11     results = tangan.process(imgRGB)
12     if results.multi_hand_landmarks:
13         for titiktangan in results.multi_hand_landmarks:
14             mediapipe draw.draw_landmarks(image, titiktangan, mediapipehand.HAND_CONNECTIONS)
15             for id, titik in enumerate(titiktangan.landmark):
16                 print (id)
17                 print (titik.x)
18                 print (titik.y)
19     cv2.imshow("webcam", image)
20     cv2.waitKey(10)
21     if cv2.waitKey(10) & 0xFF == ord('q'):
22         break # Tutup webcam dan jendela tampilan q ditekan
23     capture.release()
24     cv2.destroyAllWindows()
```

C. Latihan Soal A



```
Percobaan A.py Percobaan B.py latihan Soal A.py x
1 import cv2
2 import mediapipe as mp
3
4 mp_hands = mp.solutions.hands
5 hands = mp_hands.Hands(
6     static_image_mode=False,
7     max_num_hands=2,
8     min_detection_confidence=0.8,
9     min_tracking_confidence=0.8
10 )
11 mp_draw = mp.solutions.drawing_utils
12
13 cap = cv2.VideoCapture(0)
14
15 while True:
16     ret, frame = cap.read()
17     if not ret:
18         break
19
20     frame = cv2.flip(frame, 1)
21     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
22     results = hands.process(rgb)
23
24     if results.multi_hand_landmarks and results.multi_handedness:
25         for idx, hand in enumerate(results.multi_handedness):
26             label = hand.classification[0].label # kanan / kiri
27
28             if label == "Right":
29                 cv2.putText(frame, "TANGAN KANAN", (50, 50),
30                             cv2.FONT_HERSHEY_PLAIN, 3, (0, 0, 255), 3)
31             else:
32                 cv2.putText(frame, "TANGAN KIRI", (50, 50),
33                             cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 0), 3)
34
35             mp_draw.draw_landmarks(
36                 frame,
37                 results.multi_hand_landmarks[idx],
38                 mp_hands.HAND_CONNECTIONS
39             )
40
41     cv2.imshow("Deteksi Kanan & Kiri", frame)
42
43     if cv2.waitKey(1) & 0xFF == 28:
44         break
45
46 cap.release()
47 cv2.destroyAllWindows()
48
```

D. Latihan Soal B

Deteksi Depan & Belakang



Deteksi Depan & Belakang



```
Percobaan A.py Percobaan B.py latihan Soal A.py latihan Soal B.py x
1 import cv2
2 import mediapipe as mp
3
4 mp_hands = mp.solutions.hands
5 hands = mp_hands.Hands(
6     static_image_mode=False,
7     max_num_hands=1,
8     min_detection_confidence=0.8,
9     min_tracking_confidence=0.8
10 )
11 mp_draw = mp.solutions.drawing_utils
12
13 cap = cv2.VideoCapture(0)
14
15 while True:
16     ret, frame = cap.read()
17     if not ret:
18         break
19
20     frame = cv2.flip(frame, 1)
21     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
22     results = hands.process(rgb)
23
24     if results.multi_hand_landmarks:
25         for landmarks in results.multi_hand_landmarks:
26             wrist = landmarks.landmark[0]
27             index_mcp = landmarks.landmark[5]
28             pinky_mcp = landmarks.landmark[17]
29
30             # Deteksi telapak / punggung
31             if index_mcp.x < pinky_mcp.x:
32                 posisi = "TELAPAK (DEPAN)"
33                 warna = (0, 255, 0)
34             else:
35                 posisi = "PUNGGUNG (BELAKANG)"
36                 warna = (0, 255, 255)
37
38             cv2.putText(frame, posisi, (50, 50),
39                         cv2.FONT_HERSHEY_PLAIN, 3, warna, 3)
40
41             mp_draw.draw_landmarks(
42                 frame,
43                 landmarks,
44                 mp_hands.HAND_CONNECTIONS
45             )
46
47             cv2.imshow("Deteksi Depan & Belakang", frame)
48
49             if cv2.waitKey(1) & 0xFF == 28:
50                 break
51
52 cap.release()
53 cv2.destroyAllWindows()
54
```

5. Analisis Program

- Percobaan A

Pada Percobaan A, program bertujuan untuk mendeteksi keberadaan tangan secara real-time menggunakan library OpenCV dan MediaPipe Hands. Sistem bekerja dengan menangkap frame video dari kamera, kemudian melakukan konversi warna dari BGR ke RGB sebelum diproses oleh model MediaPipe. Model tersebut akan mengidentifikasi area tangan yang terdeteksi. Hasil percobaan menunjukkan bahwa sistem bisa mendeteksi tangan dengan baik dalam kondisi pencahayaan yang memadai. Proses deteksi berjalan secara real-time dan responsif. Namun, akurasi dapat menurun apabila pencahayaan kurang optimal atau tangan bergerak terlalu cepat.

- Percobaan B

Pada Percobaan B, program dikembangkan lebih lanjut dengan menampilkan 21 titik landmark pada tangan yang terdeteksi. Setiap landmark memiliki koordinat (x, y, z) yang merepresentasikan posisi sendi dan ujung jari. Titik-titik tersebut kemudian divisualisasikan dalam bentuk garis dan titik untuk menunjukkan struktur tangan secara lengkap. Hasil analisis menunjukkan bahwa sistem mampu melacak pergerakan tangan secara stabil dan akurat dalam waktu nyata. Dengan adanya landmark ini, program dapat dikembangkan untuk pengenalan gestur atau interaksi berbasis gerakan tangan.

- Latihan Soal A

Program ini digunakan untuk mendeteksi posisi tangan kanan dan kiri secara real-time menggunakan OpenCV dan MediaPipe. Inisialisasi dilakukan dengan mengaktifkan MediaPipe Hands dengan parameter `max_num_hands=2` sehingga sistem dapat mendeteksi maksimal dua tangan sekaligus. Nilai `min_detection_confidence` dan `min_tracking_confidence` sebesar 0,8 digunakan untuk meningkatkan tingkat keakuratan deteksi dan pelacakan. Citra diambil dari webcam melalui `VideoCapture(0)` kemudian setiap frame dibalik (flip) agar tampilan seperti cermin dan dikonversi dari format BGR ke RGB sebelum diproses oleh MediaPipe. Setelah proses deteksi, sistem memeriksa apakah terdapat landmark tangan dan informasi handedness (kanan/kiri). Penentuan tangan kanan atau kiri diperoleh dari `results.multi_handedness`, di mana label “Right” menunjukkan tangan kanan dan “Left” menunjukkan tangan kiri. Hasil klasifikasi ditampilkan pada layar menggunakan fungsi `cv2.putText()`, dan landmark tangan digambar menggunakan `drawing_utils` untuk memperjelas posisi titik-titik tangan.

- Latihan Soal B

Program ini dibuat untuk mendeteksi orientasi tangan, yaitu telapak (depan) dan punggung (belakang), menggunakan library OpenCV dan MediaPipe pada bahasa pemrograman Python. Pada tahap awal dilakukan inisialisasi modul MediaPipe Hands dengan parameter `static_image_mode=False` agar dapat digunakan secara real-time melalui webcam, `max_num_hands=1` untuk membatasi deteksi satu tangan, serta nilai `min_detection_confidence` dan `min_tracking_confidence` sebesar 0,8 untuk meningkatkan akurasi deteksi dan pelacakan. Citra diperoleh dari

webcam menggunakan VideoCapture(0) kemudian setiap frame yang tertangkap dikonversi dari format warna BGR ke RGB karena MediaPipe hanya dapat memproses citra dalam format RGB. Setelah proses konversi, citra dianalisis untuk mendeteksi landmark tangan. Jika tangan terdeteksi, sistem akan mengambil beberapa titik penting, yaitu landmark pergelangan tangan (0), pangkal jari telunjuk (5), dan pangkal jari kelingking (17). Penentuan posisi tangan dilakukan dengan membandingkan nilai koordinat sumbu x antara landmark telunjuk dan kelingking. Apabila nilai x telunjuk lebih kecil dari kelingking, maka tangan dianggap menghadap depan (telapak). Sebaliknya, jika nilai x telunjuk lebih besar, maka tangan dianggap menghadap belakang (punggung). Hasil deteksi ditampilkan pada layar webcam menggunakan fungsi putText, dan seluruh titik landmark digambarkan menggunakan drawing_utils.

6. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi deteksi dan pelacakan tangan menggunakan MediaPipe Hands dan OpenCV pada bahasa pemrograman Python dapat berjalan dengan baik secara real-time. Pada Percobaan A, sistem mampu mendeteksi keberadaan tangan melalui webcam dengan cukup akurat dalam kondisi pencahayaan yang memadai. Pada Percobaan B, sistem berhasil menampilkan 21 titik landmark yang merepresentasikan struktur tangan secara detail sehingga memungkinkan analisis pergerakan tangan. Pada Latihan Soal A, program mampu mengidentifikasi posisi tangan kanan dan kiri berdasarkan informasi handedness yang dihasilkan oleh MediaPipe. Sedangkan pada Latihan Soal B, sistem berhasil menentukan orientasi tangan (telapak atau punggung) dengan membandingkan koordinat landmark tertentu.

7. Saran

- Sebaiknya praktikum dilakukan pada kondisi pencahayaan yang cukup agar proses deteksi dan pelacakan tangan berjalan lebih optimal
- Untuk meningkatkan akurasi, dapat digunakan kamera dengan resolusi yang lebih tinggi serta pengaturan parameter deteksi yang lebih sesuai.
- Pengembangan selanjutnya dapat dilakukan dengan menambahkan fitur klasifikasi gestur yang lebih kompleks menggunakan metode machine learning tambahan.

8. Referensi

- Belajar Python– Situs Open Source Tutorial Pemrograman Python Bahasa Indonesia <https://belajarpython.com>